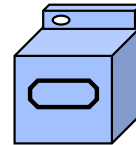
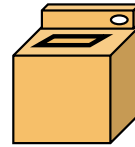
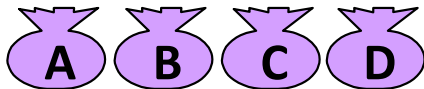


Pipelining

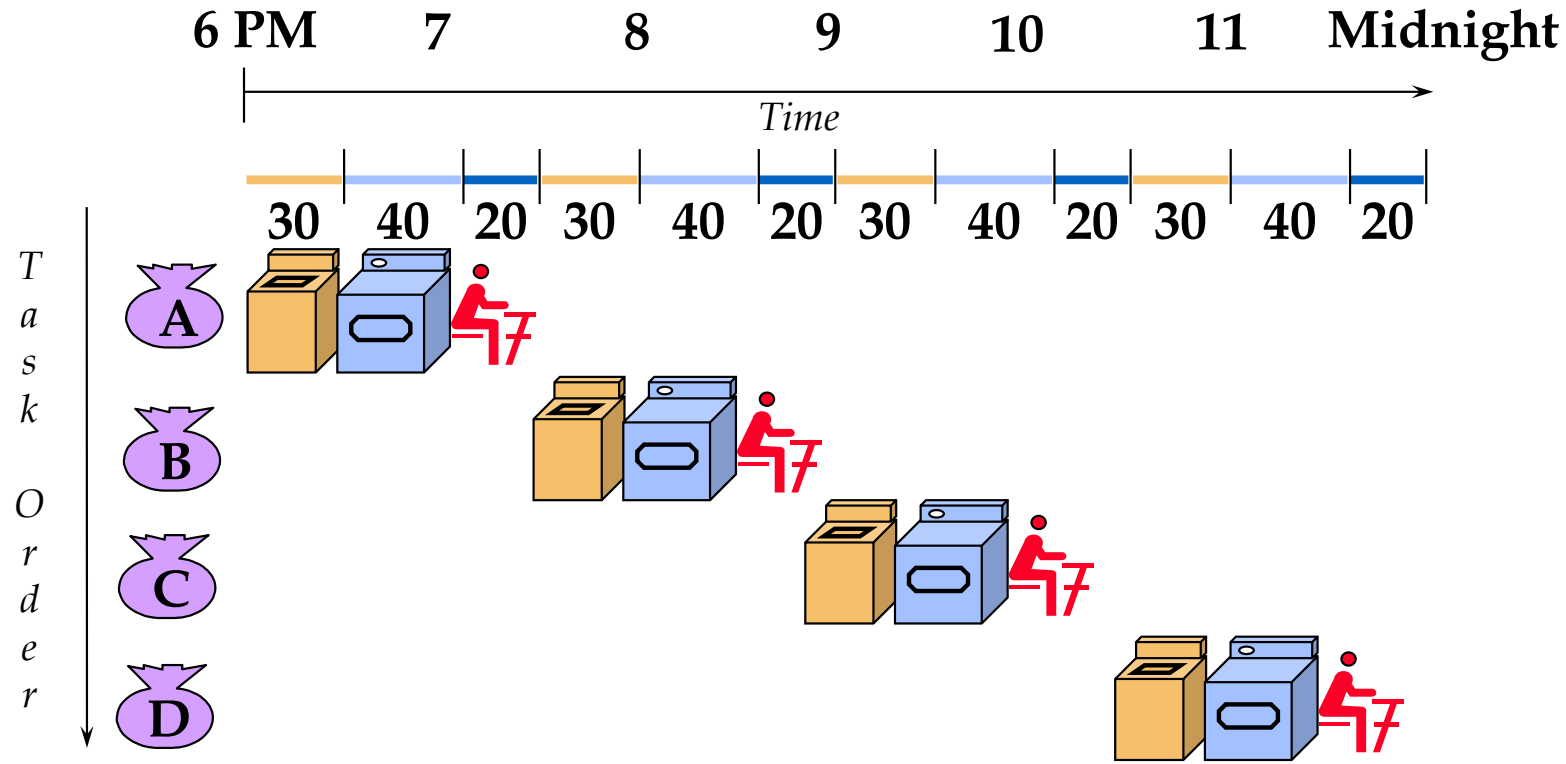
`mano@cs.auckland.ac.nz`

Pipelining

- Pipelining provides an efficient way of executing multiple tasks concurrently
- The laundry example
 - We have four loads of laundry to do. This consists of washing, drying, and folding.
 - Washer takes 30 minutes
 - Dryer takes 40 minutes
 - Folding takes 20 minutes

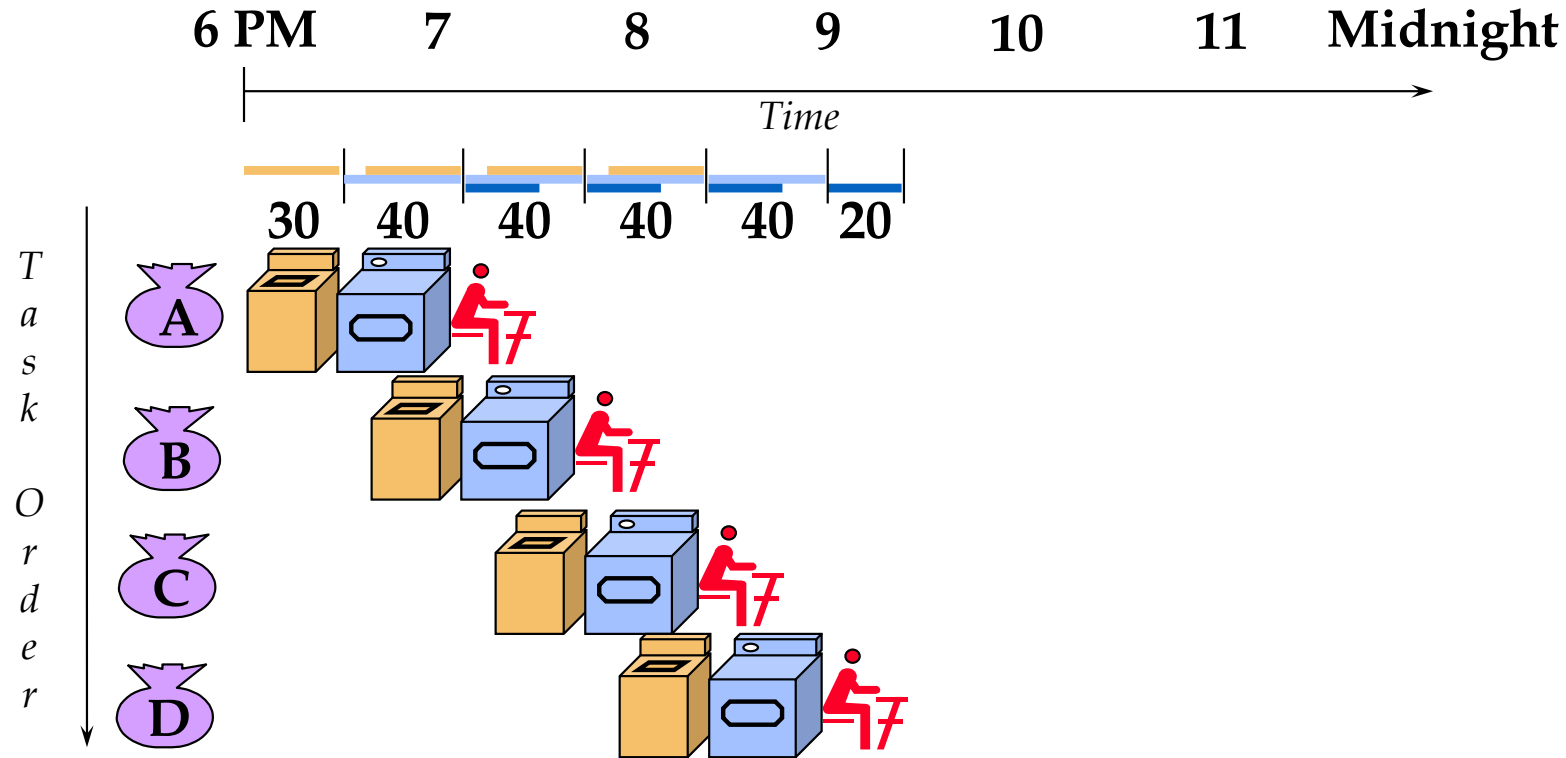


Sequential Laundry



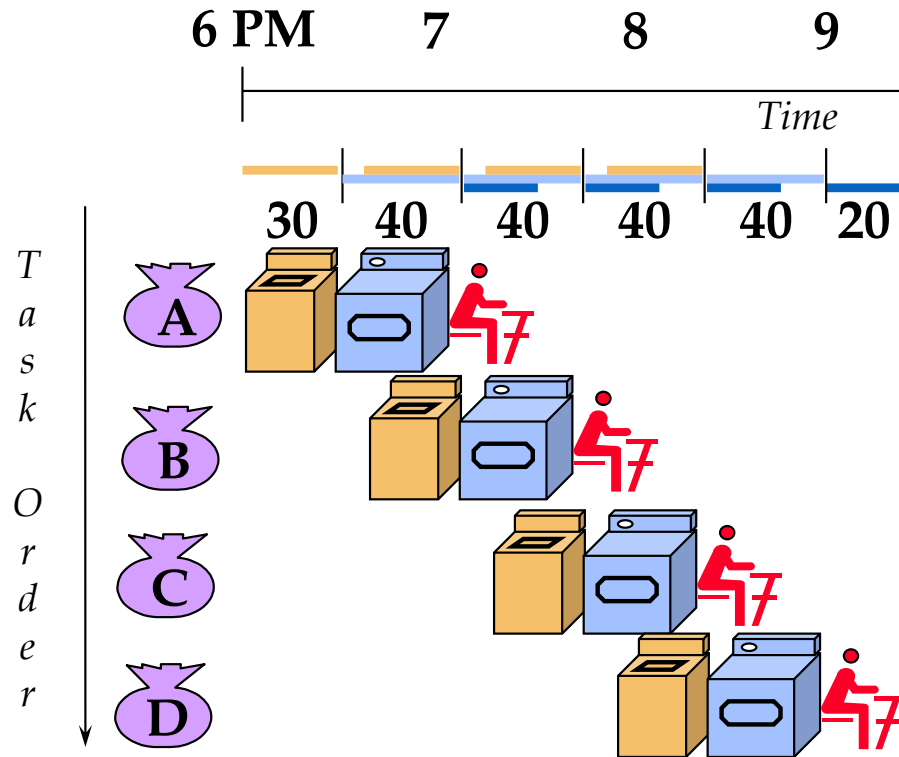
Sequential laundry takes 6 hours for 4 loads

Pipelined Laundry



Pipelined laundry takes 3 ½ hours for 4 loads

Pipelining



Multiple tasks operating simultaneously

Pipelining doesn't help *latency* of single task, it helps *throughput* of entire workload

Pipeline rate limited by the *slowest* pipeline stage

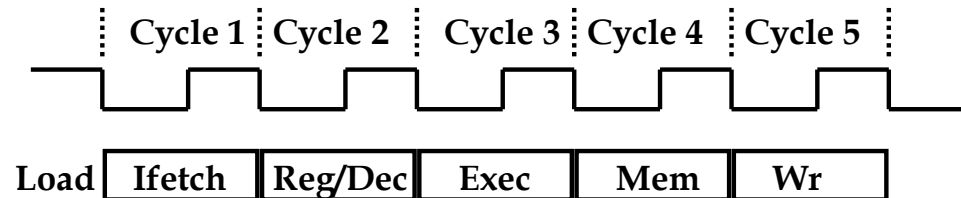
Potential speedup = number pipe stages

Unbalanced lengths of pipe stages reduces speedup

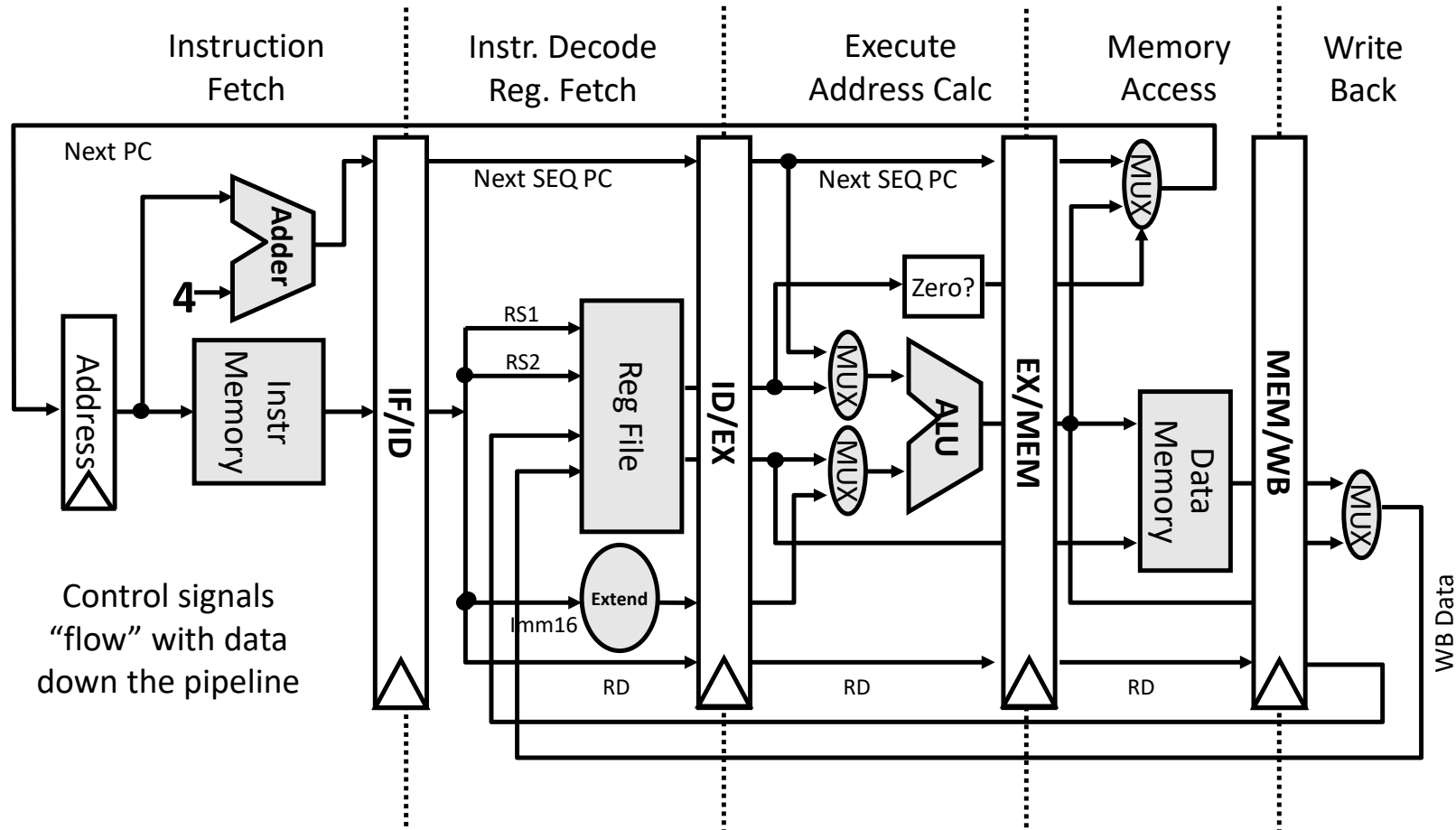
Also, need time to “fill” and “drain” the pipeline.

The Load Instruction

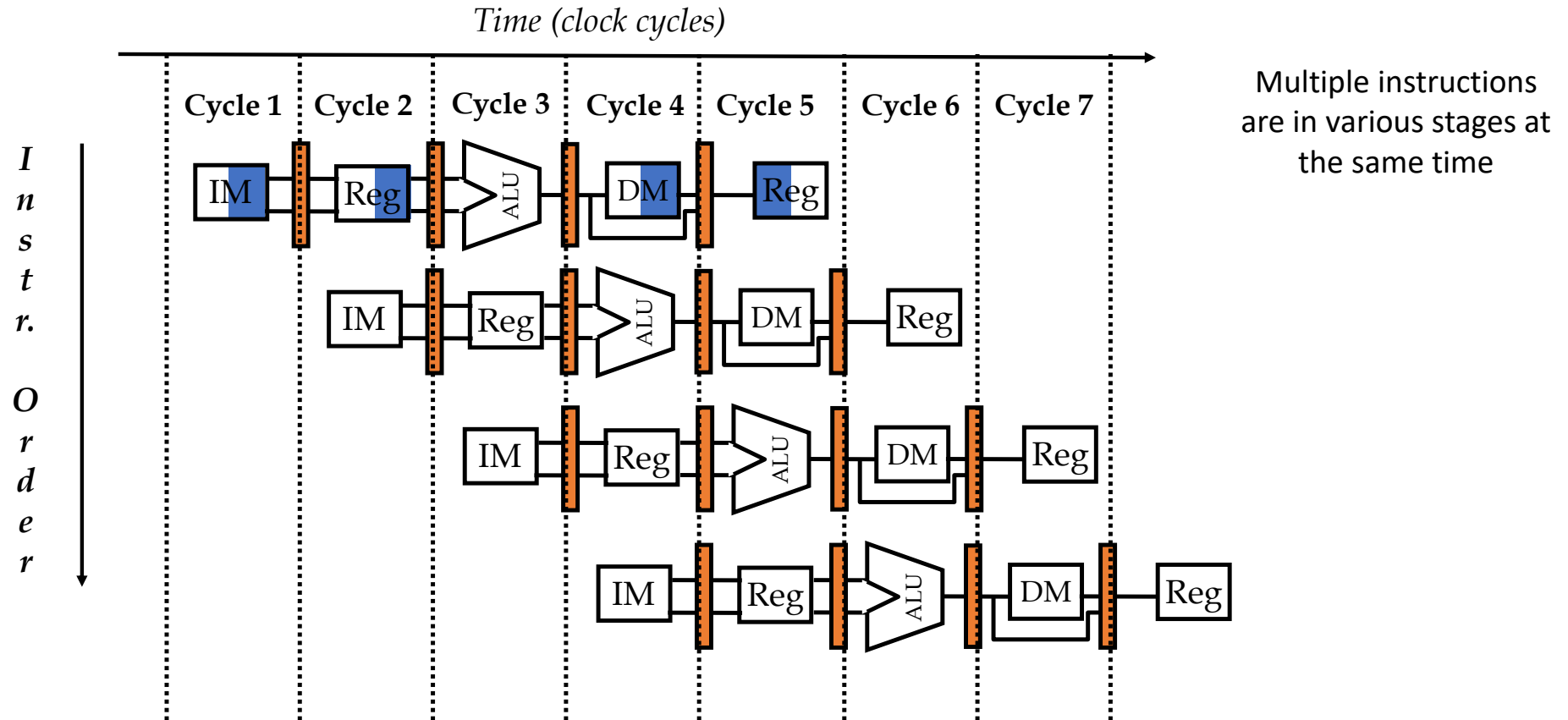
- Fetch the instruction from the instruction memory
- Register fetch and instruction decode
- Calculate the memory address (using ALU)
- Read the data from the data memory
- Write the data back to the register file



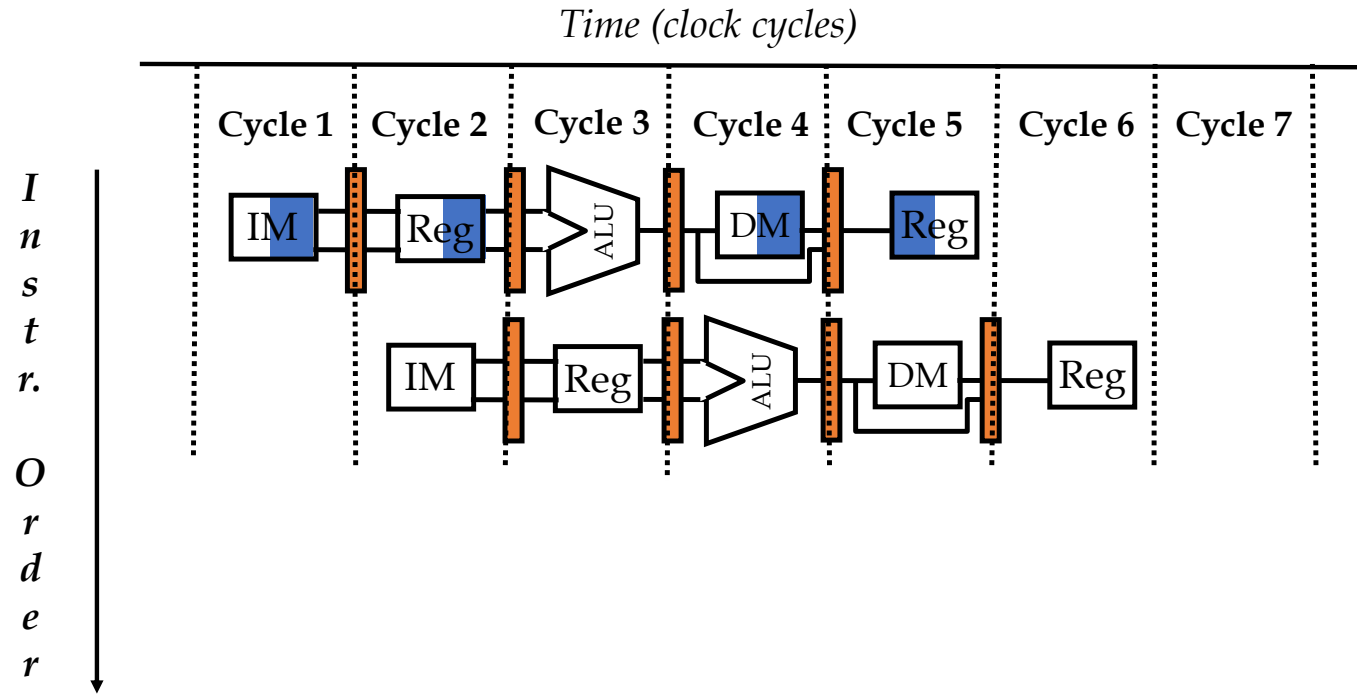
The Steps of the MIPS Datapath



Visualizing Pipelining



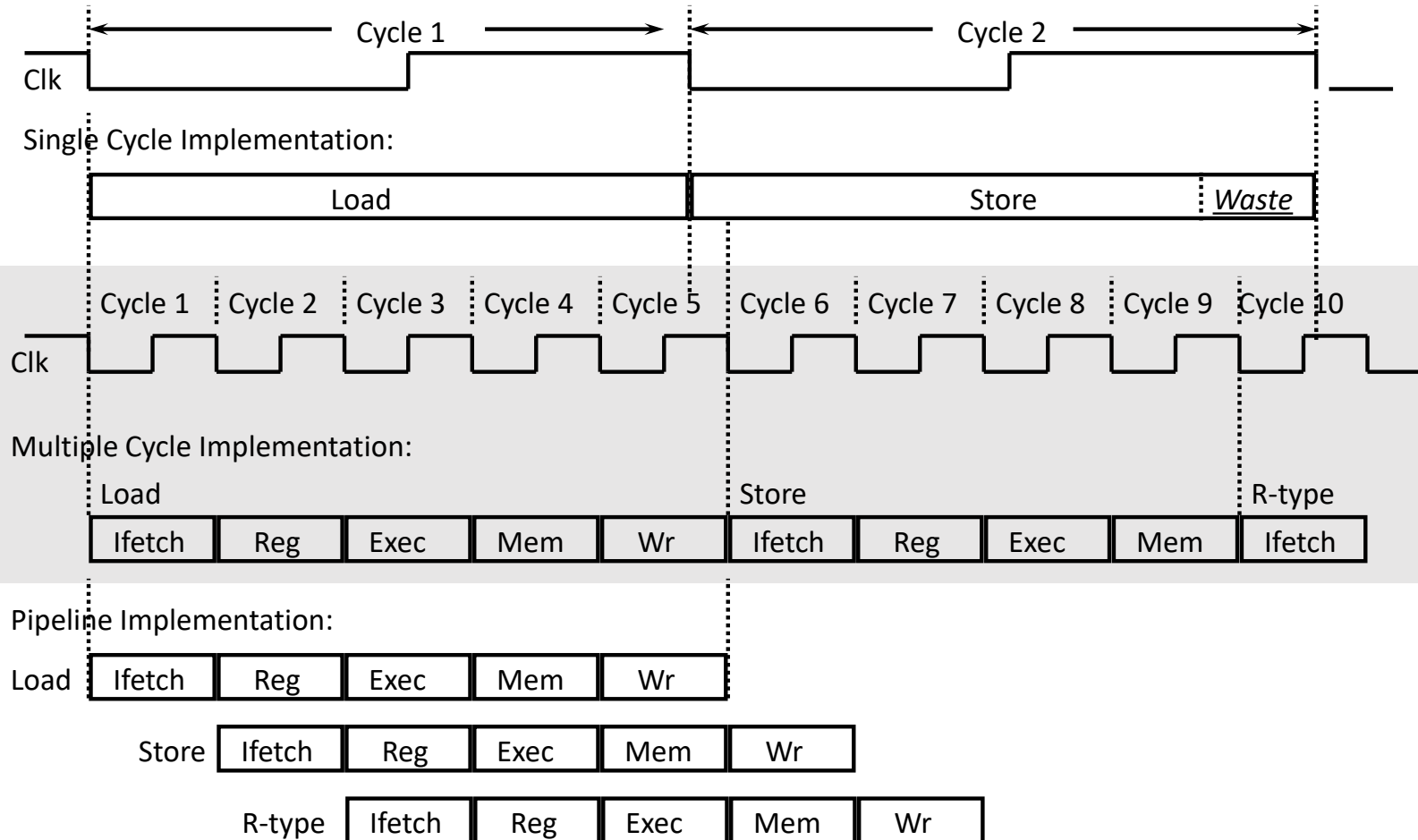
Visualizing Pipelining



Can help with answering questions such as:

1. How many cycles does it take to execute this code?
2. What is the ALU doing during cycle 4?
3. Are two instructions trying to use the same resource at the same time?

Comparison of Processor Design



Comparison of Processor Design

- Suppose we have 100 instructions to execute
 - Single cycle processor has a cycle time of 45 ns
 - Multi cycle and pipeline processors have cycle times of 10 ns
 - The multi cycle processor has a CPI of 3.8
- Single cycle processor
 - $45 \text{ ns/cycle} \times 1 \text{ CPI} \times 100 \text{ inst} = 4500 \text{ ns}$
- Multi cycle processor
 - $10 \text{ ns/cycle} \times 3.8 \text{ CPI} \times 100 \text{ inst} = 3800 \text{ ns}$
- Ideal pipelined machine
 - $10 \text{ ns/cycle} \times (1 \text{ CPI} \times 100 \text{ inst} + 4 \text{ cycle "drain"}) = 1040 \text{ ns}$
- Ideal pipelined vs. single cycle speedup
 - $4500 \text{ ns} / 1040 \text{ ns} = 4.33$

Pipeline Hazards 危害

- A hazard prohibits the execution of a planned instruction in the proper cycle.
- Three forms of hazards may exist in a pipeline.
 - Structural hazard
 - Data hazard
 - Control hazard

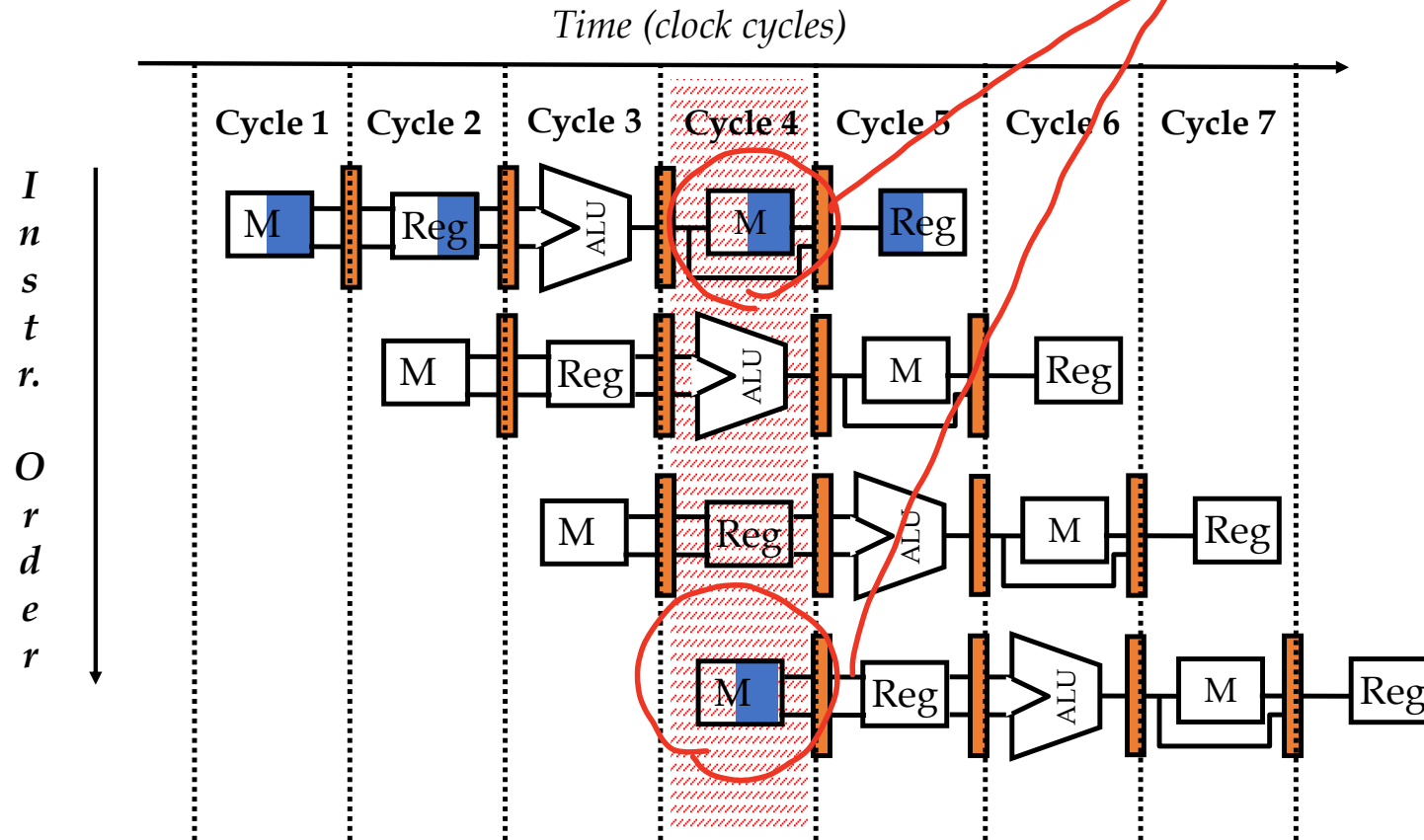
Structural Hazards

- Structural hazards arise when attempting to use the same resource at the same time.
 - If we have a combined dryer/washer, then we won't be able to use the dryer and washer simultaneously.
 - If we have a unified memory that holds both data as well as instructions, then at cycle 4 when there is data access, we won't be able to read an instruction out.

Structural hazard occurs when a planned instruction cannot execute in the proper clock cycle because the hardware cannot support the combination of instructions that are set to execute in the given clock cycle.

Structural Hazards

Conflict



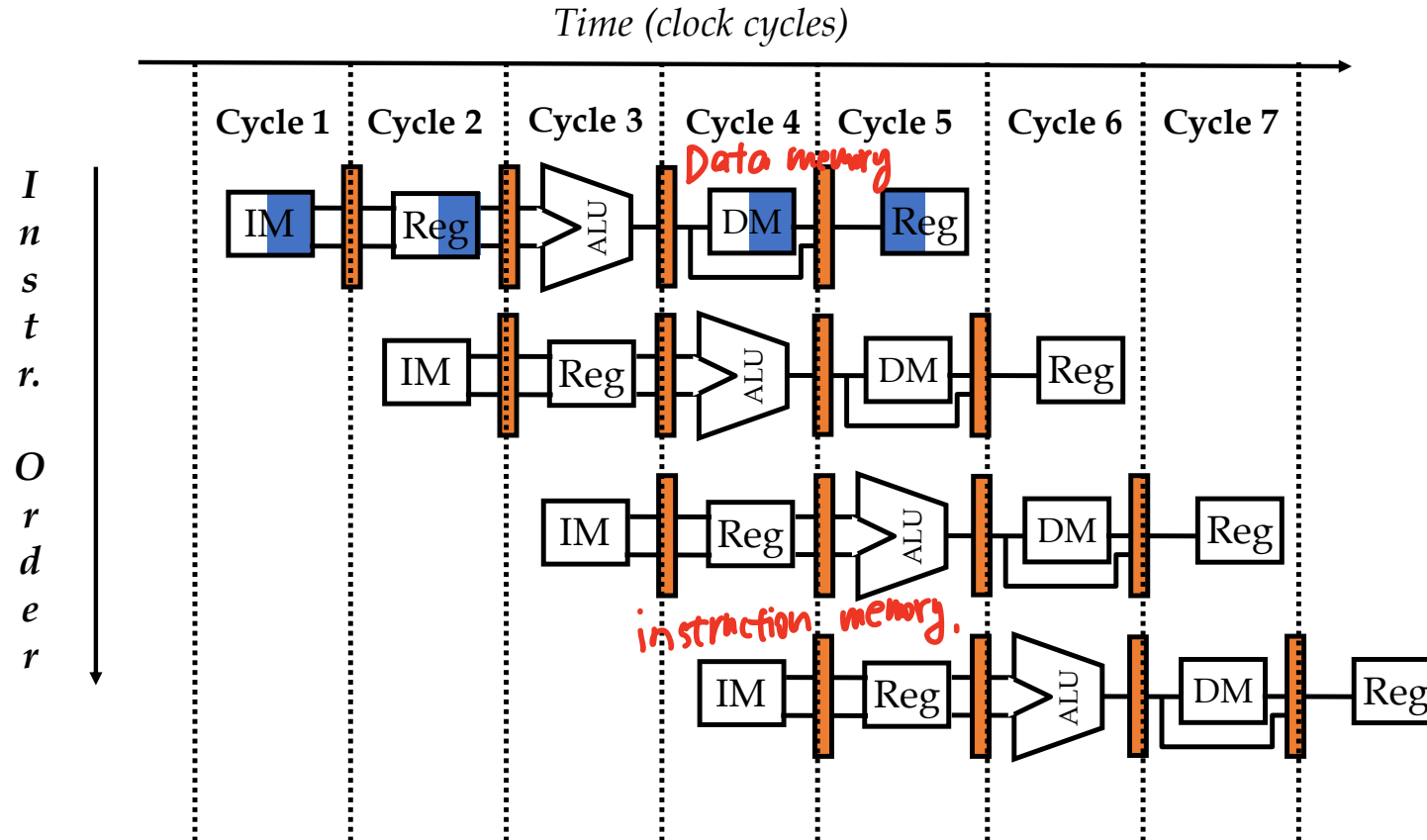
We get a structural hazard if we have a unified memory that holds both data as well as instructions.

Structural Hazards

- Solutions to structural hazards
 - Use separate instruction and data memories rather than a unified one
 - Have multi-port memory (i.e. allow several concurrent read/write accesses)
 - Stall the pipeline and wait until the current access completes

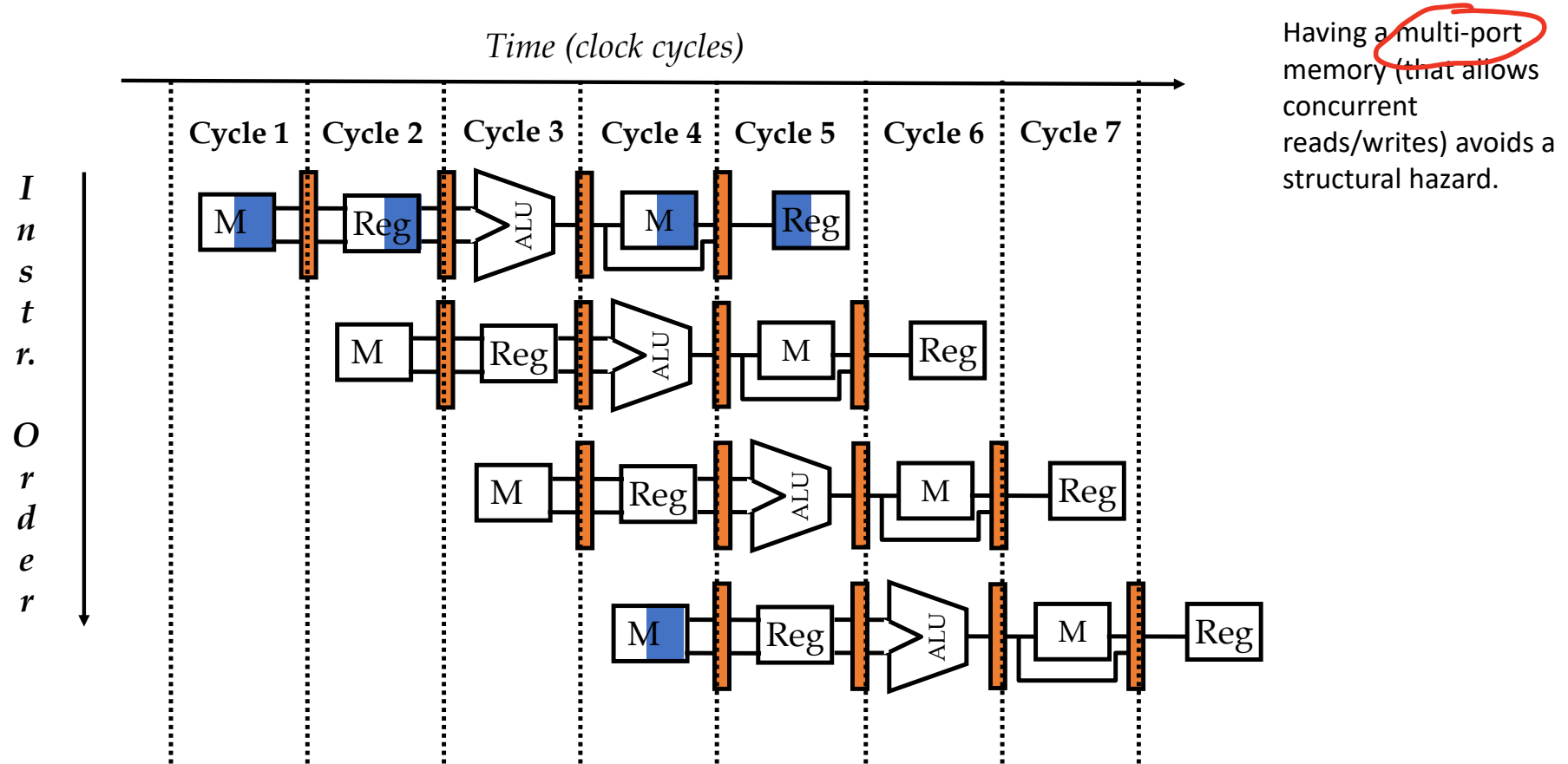
暫停

Structural Hazards



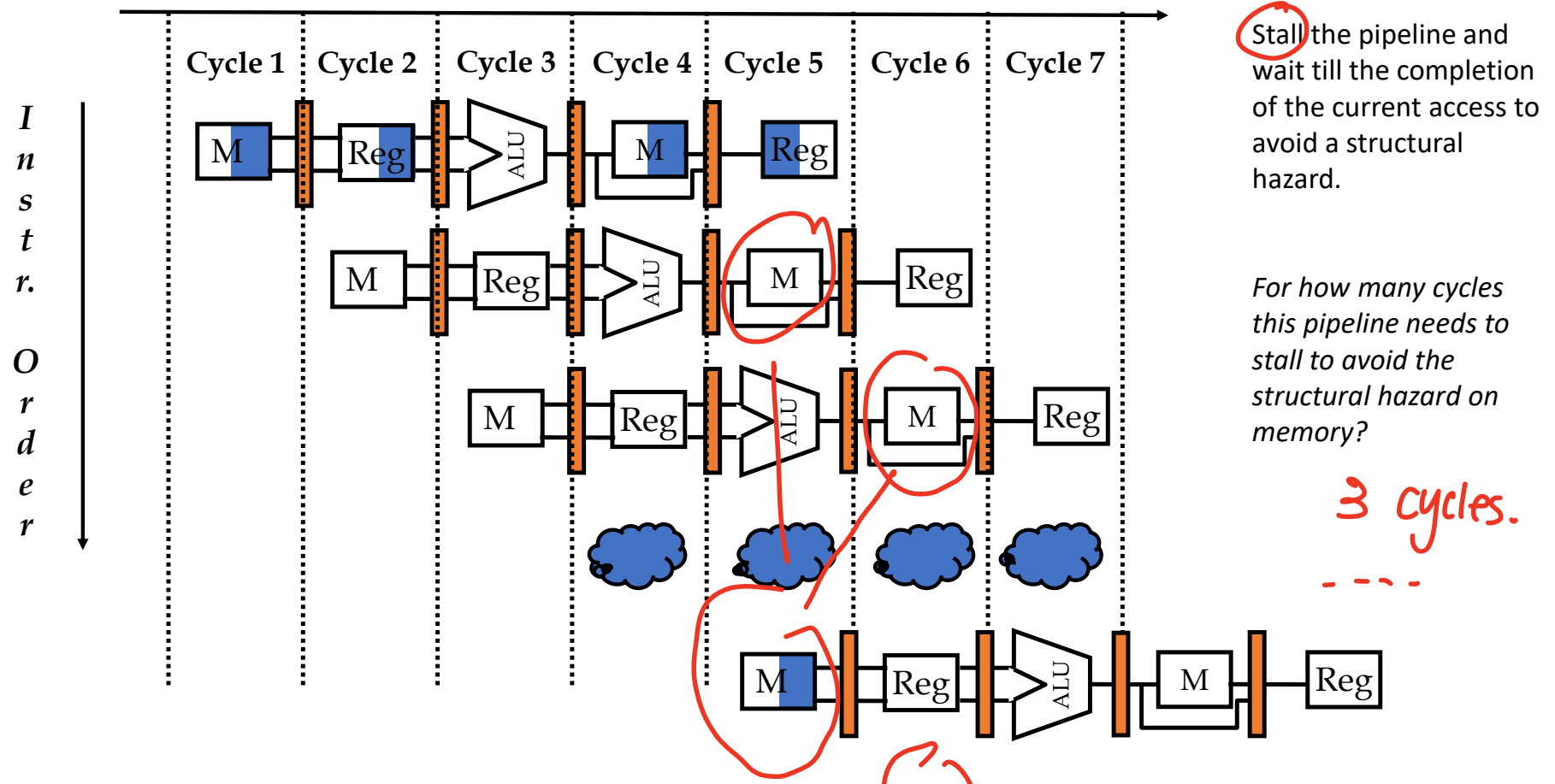
Using separate instruction and data memories rather than a unified one avoids a structural hazard.

Structural Hazards



Structural Hazards

Time (clock cycles)

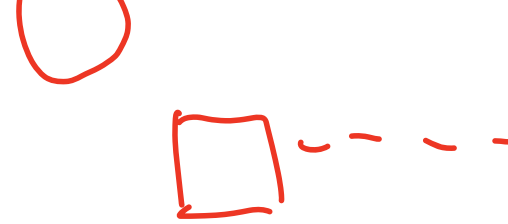


Stall the pipeline and wait till the completion of the current access to avoid a structural hazard.

For how many cycles this pipeline needs to stall to avoid the structural hazard on memory?

3 cycles.

Data Hazards



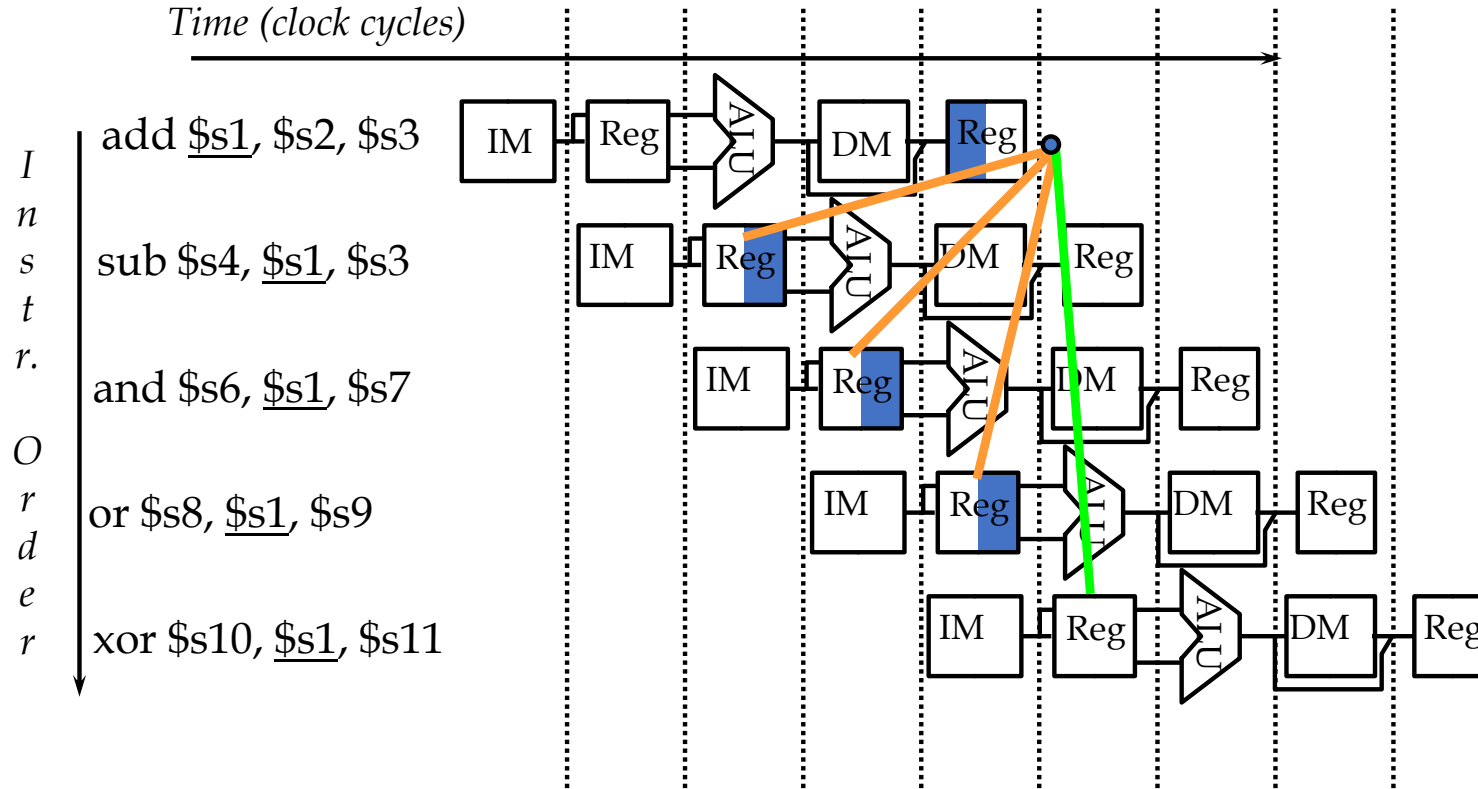
- Data hazards arise when attempting to use some data that isn't yet ready.

```
add $s0, $t0, $t1  
sub $t2, $s0, $t3
```

```
lw $s0, 20($t1)  
sub $t2, $s0, $t3
```

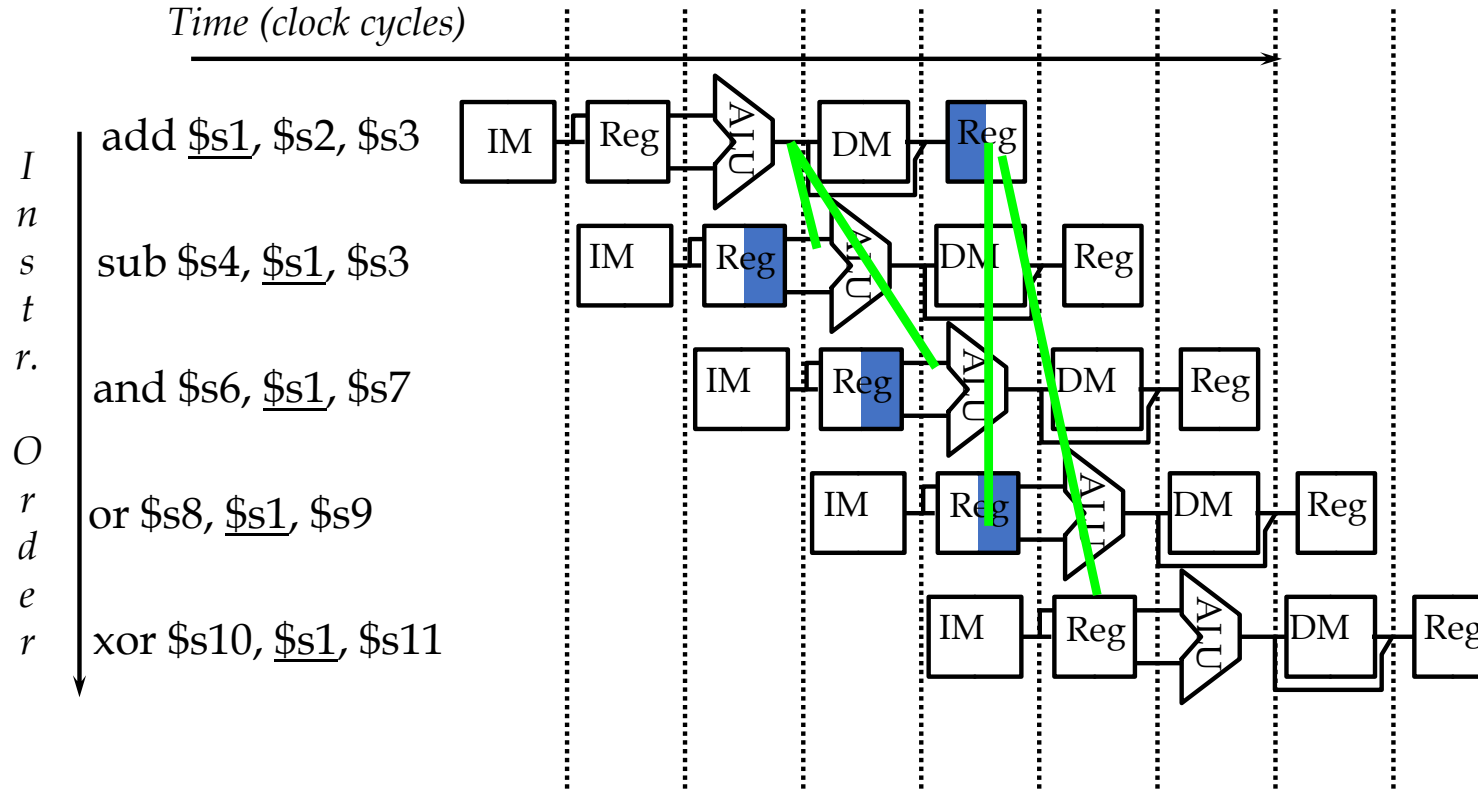
Data hazard occurs when a planned instruction cannot execute in the proper clock cycle because the data needed to execute the instruction is not available yet.

Data Hazards



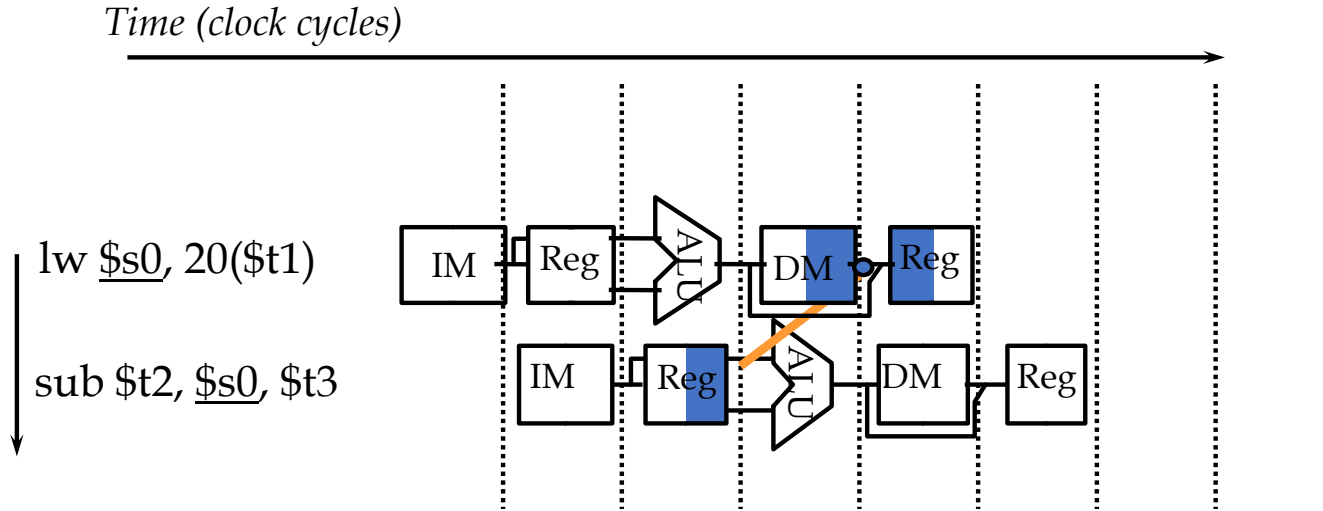
Dependencies backwards in time are hazards

Data Hazards



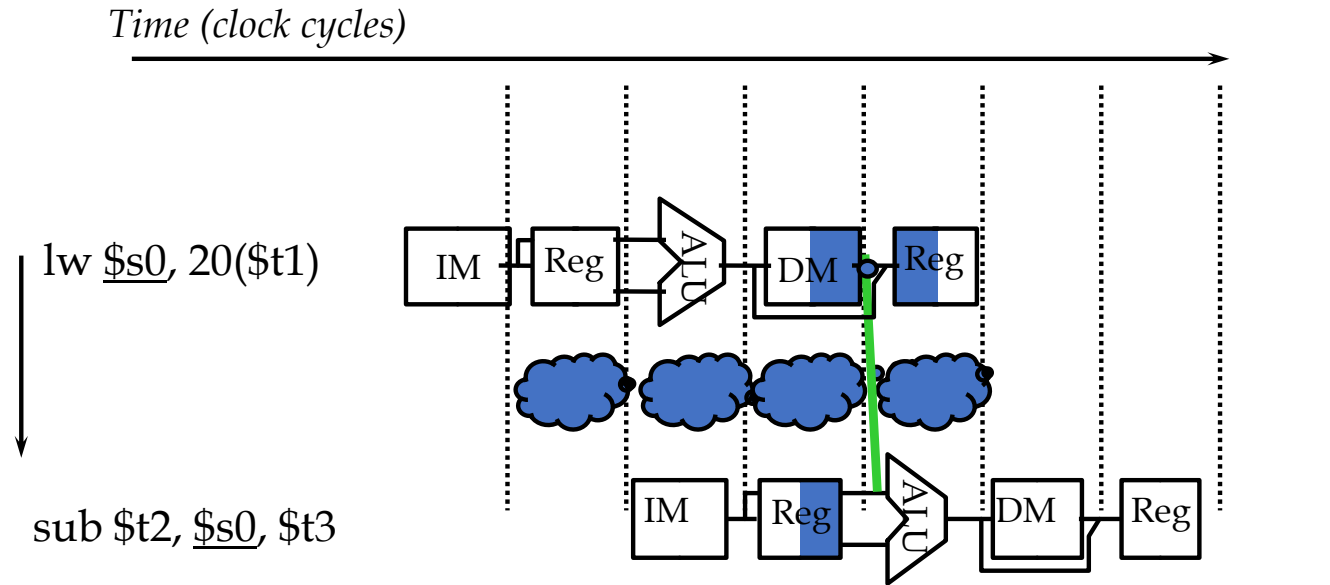
“Forward” result from one stage to another

Load-use Data Hazards

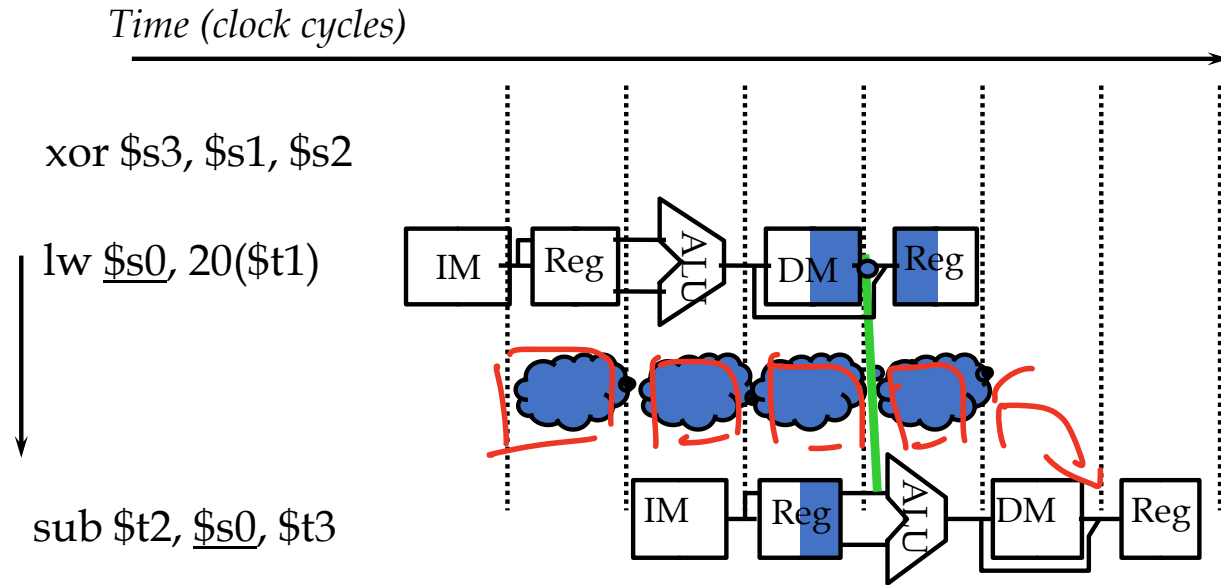


A load-use data hazard can't be solved with forwarding alone: Must delay/stall instruction dependent on loads. "*Delayed*" load.

Load-use Data Hazards



Load-use Data Hazards



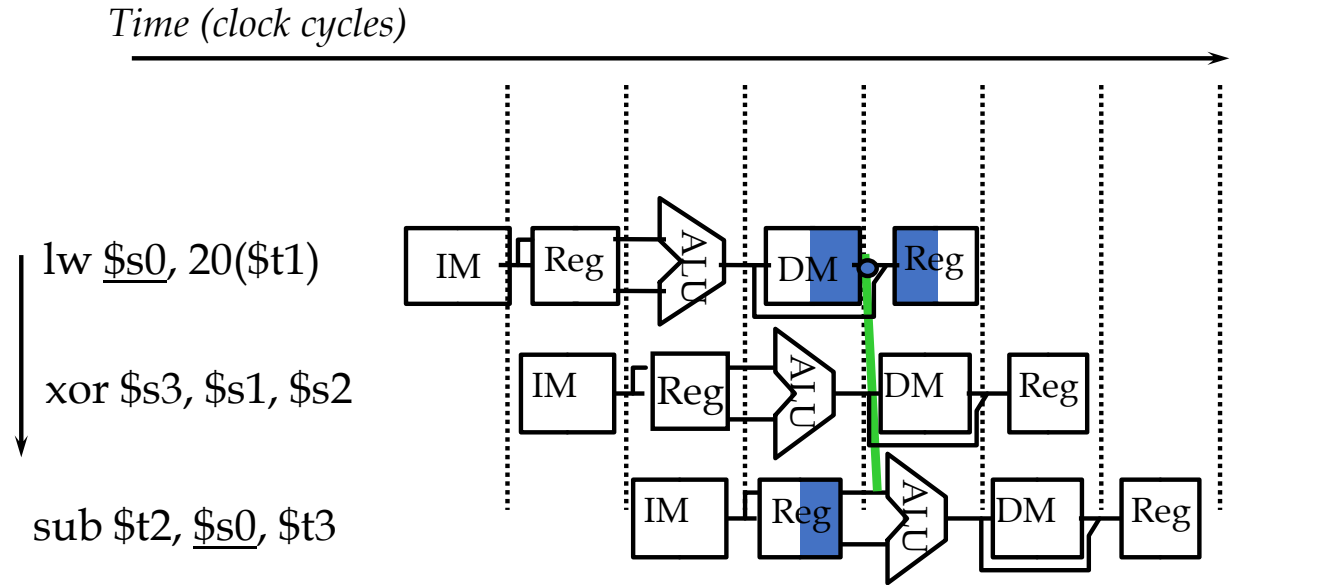
reorder?

和其它指令换序,



A load-use data hazard can't be solved with forwarding alone: Must delay/stall instruction dependent on loads. "Delayed" load.

Load-use Data Hazards



A load-use data hazard can't be solved with forwarding alone: Must delay/stall instruction dependent on loads. *"Delayed"* load.

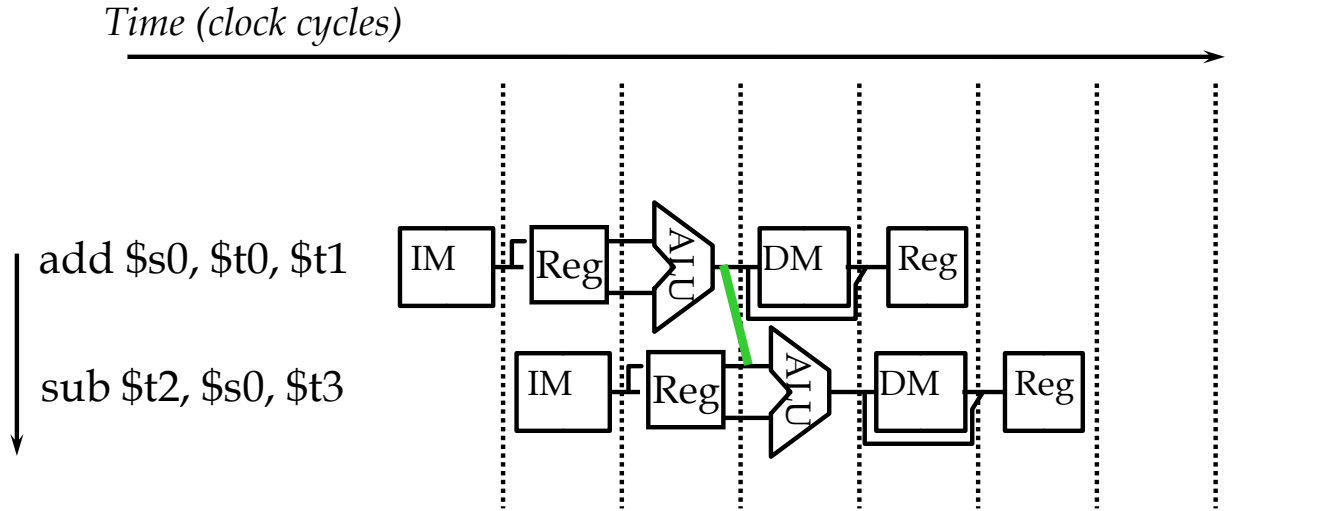
Read-After-Write Data Hazard

- Called a “dependence” in compiler terms. This hazard results from an actual need for communication.
- Error if instruction j tries to read operand *before* instruction i writes it.

i : add \$s0, \$t0, \$t1
j : sub \$t2, \$s0, \$t3

Read-After-Write hazards can be avoided by forwarding.

Read-After-Write Data Hazard



RAW hazard can be avoided by forwarding.

Write-After-Read Data Hazard

- Called an “anti-dependence” in compiler terms. This results from the reuse of the registers (e.g. *s0*).
- Error if instruction *j* tries to write operand *before* instruction *i* reads it.

i: sub \$t2, \$s0, \$t3

j: add \$s0, \$t0, \$t1

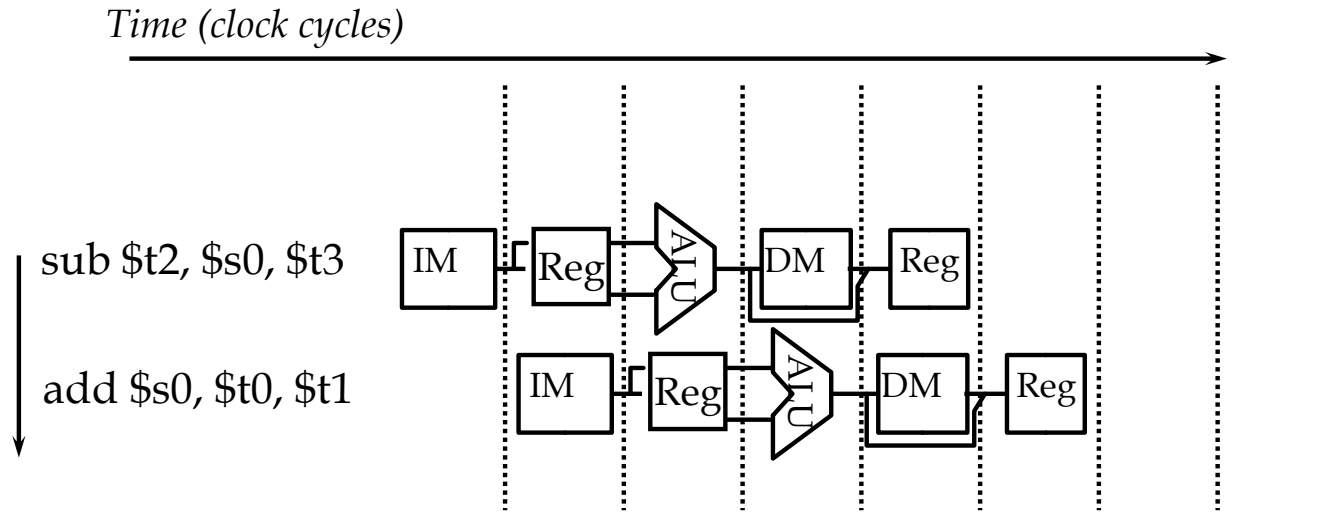
Write-After-Read Data Hazard

- Cannot happen in MIPS 5 stage pipeline because:
 - All instructions take 5 stages,
 - Reads are always in stage 2, and
 - Writes are always in stage 5

i: sub \$t2, \$s0, \$t3

j: add \$s0, \$t0, \$t1

Write-After-Read Data Hazard



WAR hazard cannot happen in the MIPS 5-stage pipeline.

Write-After-Write Data Hazard

- Called an “output dependence” in compiler terms. This too results from the reuse of the registers (e.g. *s0*).
- Error if instruction *j* tries to write operand *before* instruction *i* writes it.

i: sub \$s0, \$t2, \$t3

j: add \$s0, \$t0, \$t1

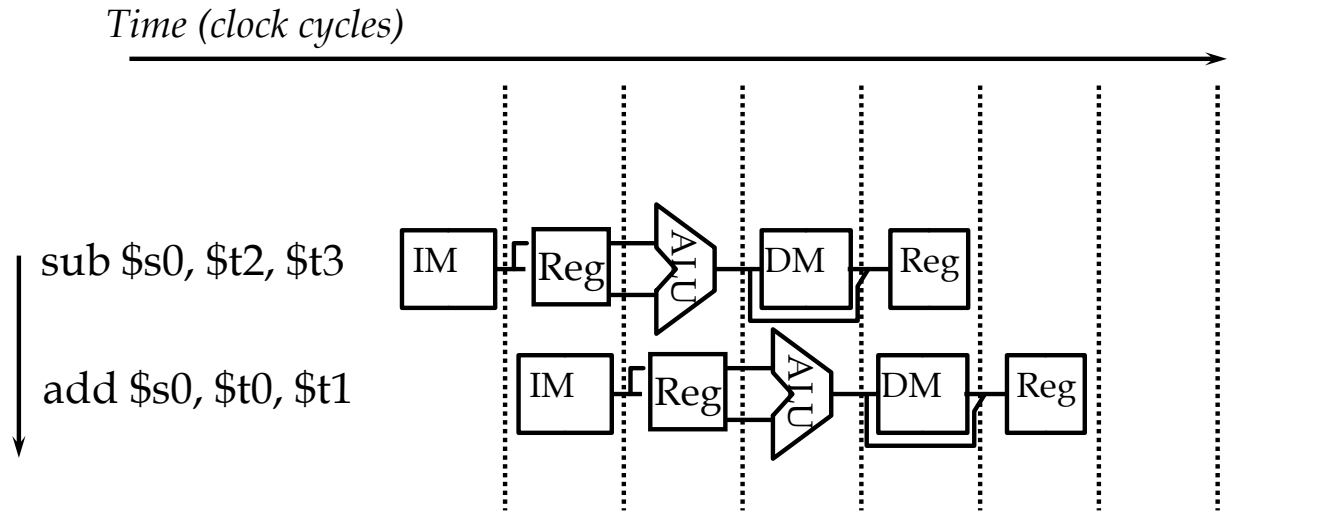
Write-After-Write Data Hazard

- Cannot happen in MIPS 5 stage pipeline because:
 - All instructions take 5 stages, and
 - Writes are always in stage 5

i: sub \$s0, \$t2, \$t3

j: add \$s0, \$t0, \$t1

Write-After-Write Data Hazard



WAW hazard cannot happen in the MIPS 5-stage pipeline.

Control Hazards

- Control hazards arise when attempting to make a decision before the condition is evaluated.
 - Also called a *branch* hazard

```
add $s0, $t0, $t1  
beq $t2, $t3, Label  
or $s3, $s4, $s5
```

Control hazard occurs when a planned instruction cannot execute in the proper clock cycle because the instruction that is fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

Control Hazards

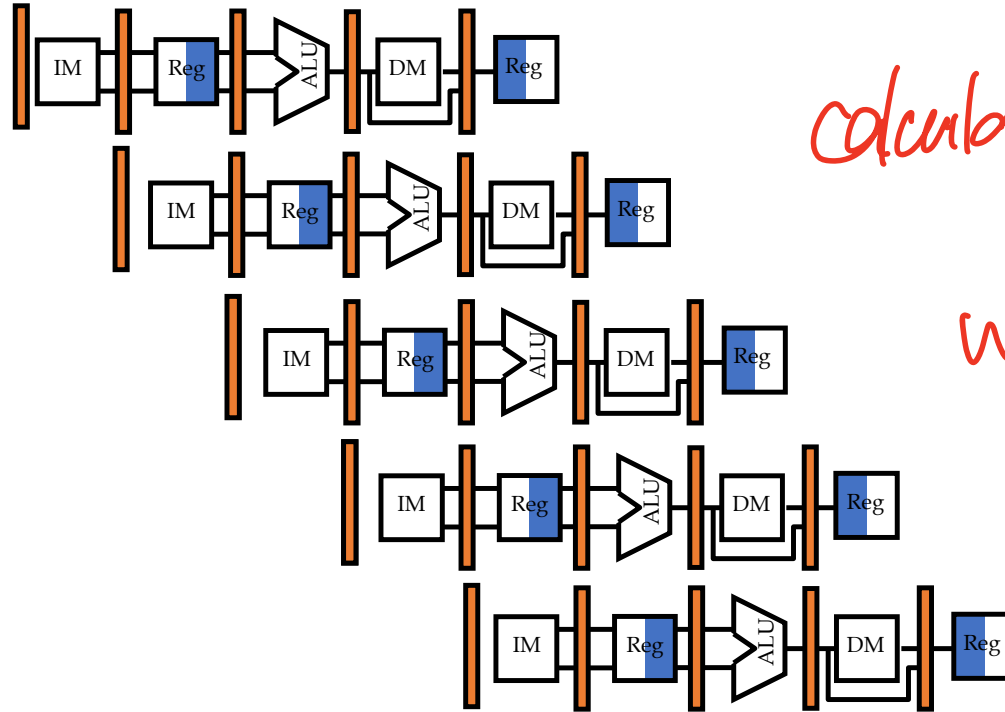
12: beq \$s1, \$s3, 36

16: and \$s2, \$s3, \$s5

20: or \$s6, \$s1, \$s7

24: add \$s8, \$s1, \$s9

36: xor \$s10, \$s1, \$s11



calculate condition

where you go - -

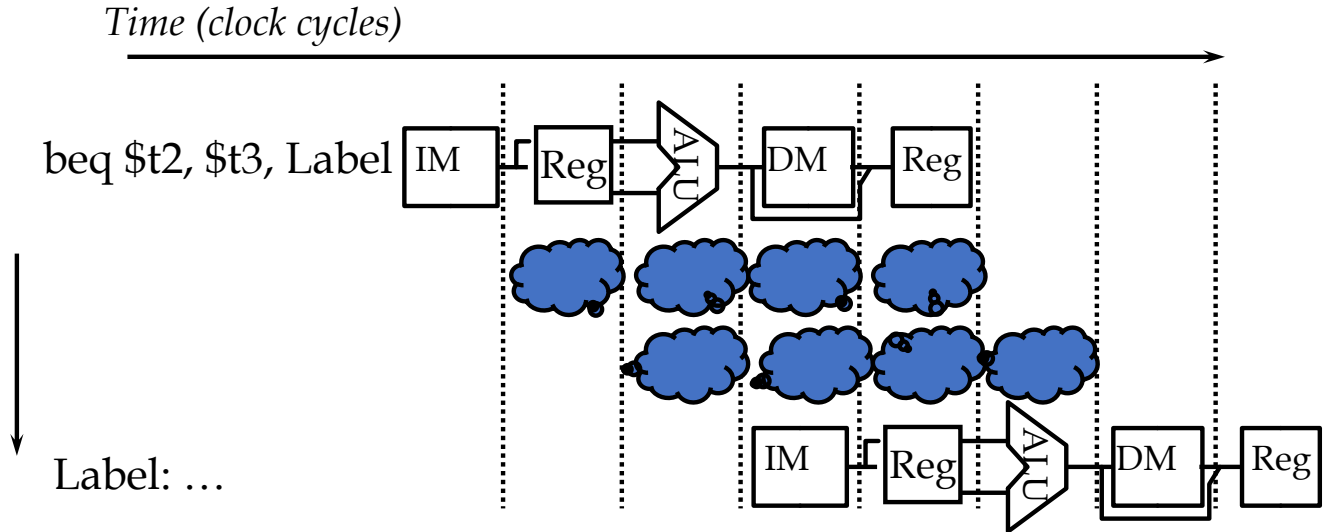
Control Hazards

- Solutions to control hazards
 - Stall until the decision is clear
 - Determine sooner if branch is taken or not *and* the taken branch address
- MIPS R10K solution
 - Move Zero test to ID/RF stage
 - Adder to calculate new PC in ID/RF stage

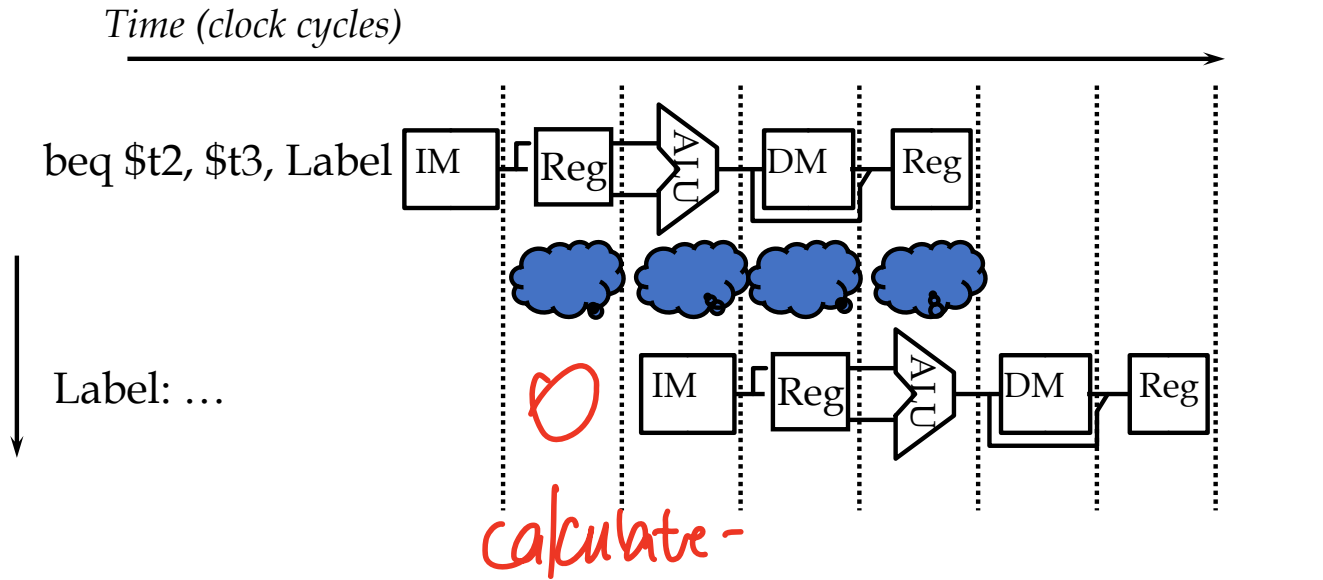
in the first stage.

other hardware calculate.

Control Hazards



Control Hazards



If the zero test and branch target calculations are done in ID/RF stage, then we have a single bubble.

branch

all calculate

Control Hazards

non branch

not cost extra
time

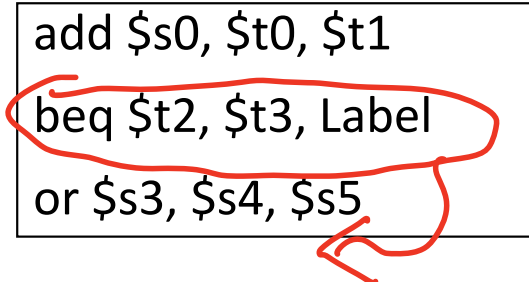
- Solutions to control hazards

- Predict the branch and roll back if the prediction is incorrect
- Rolling back involves making the predicted instruction a NOP: ensure no results of execution is committed (e.g. de-assert *MemWrite* and *RegWrite* for non-branches).

Control Hazards

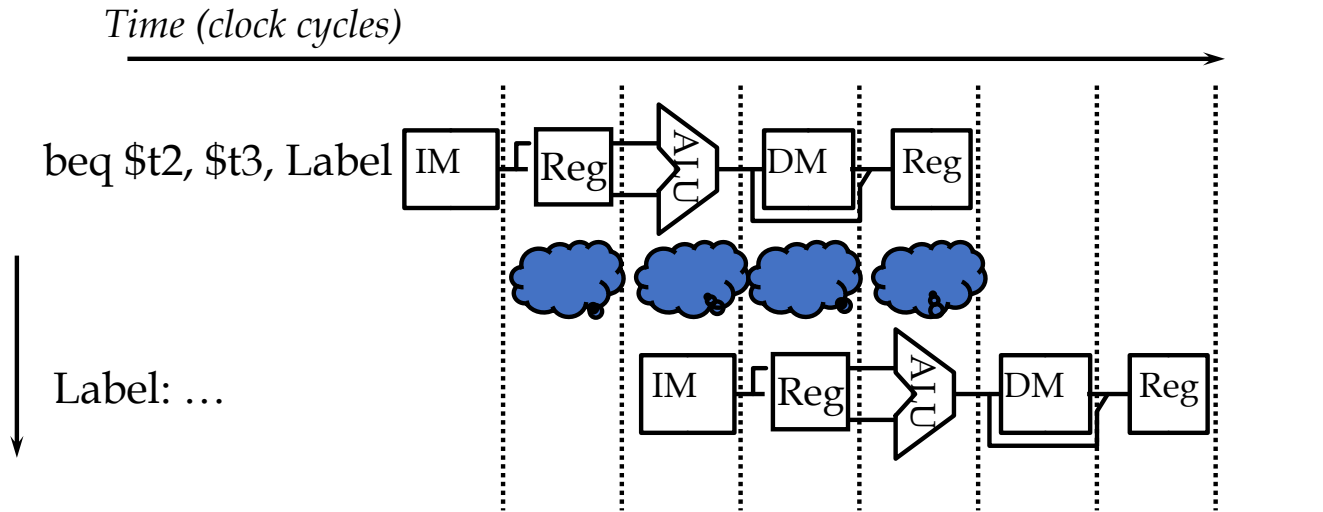
- Solutions to control hazards
 - Redefine the branch behaviour: declare that the branch takes place after executing the instruction following the branch.
 - Called “delayed branch”

```
add $s0, $t0, $t1  
beq $t2, $t3, Label  
or $s3, $s4, $s5
```



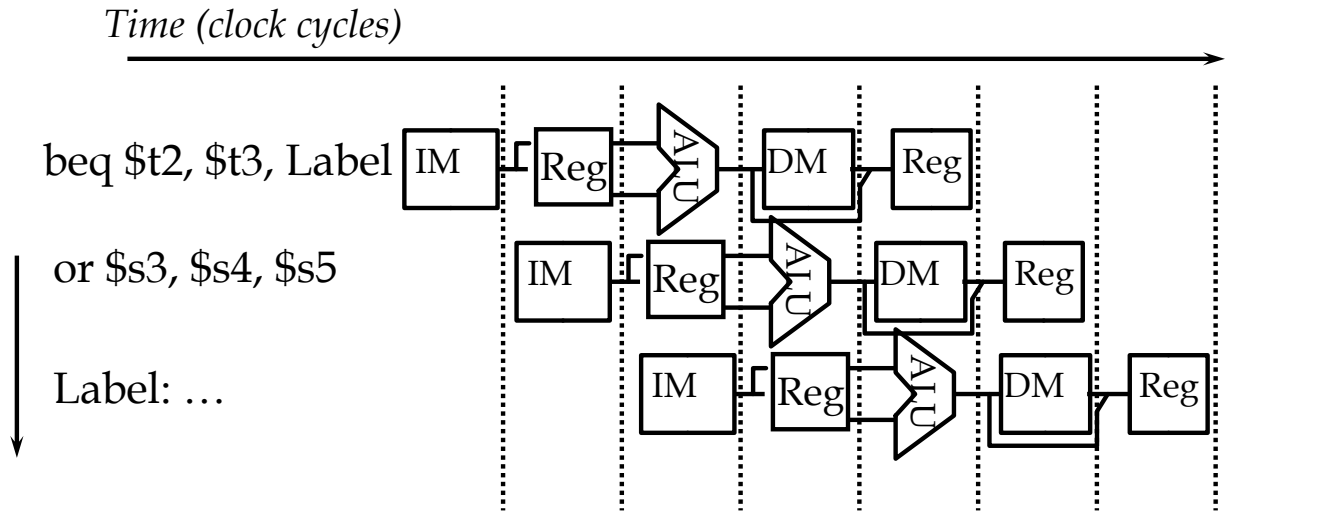
delayed.

Control Hazards



Do the OR instruction where the bubble is!

Control Hazards



Do the OR instruction where the bubble is!

Pipeline Hazards

- Avoid some “by design”
 - Eliminate WAR by always fetching operands early (during operand fetch stage) in pipe
 - Eliminate WAW by doing all write-backs in order (e.g. always in the last stage)
- Detect and resolve remaining ones
 - Stall or forward (if possible)

Hazard Detection

- Suppose instruction i is about to be issued and a predecessor instruction j is in the instruction pipeline.
 - A RAW hazard exists on register r if $r \in R_r(i) \cap R_w(j)$
 - A WAR hazard exists on register r if $r \in R_w(i) \cap R_r(j)$
 - A WAW hazard exists on register r if $r \in R_w(i) \cap R_w(j)$

$R_r(i)/R_w(i)$: Set of registers read/written by instruction i .

Further Work

- Read Chapter 4 of the book: Computer Organization and Design – The Hardware Software Interface RISC-V edition, 2nd edition (2020)
 - See what changes are necessary to the datapath to support forwarding.
 - Also read Appendix C (*Mapping control to hardware*).

¿?